

华东师范大学数据科学与工程学院实验报告

课程名称：AI 基础

年级：2024 级

上机实践日期：

2026 年 4 月 10 日

指导教师：杨彬

姓名：唐一嘉

上机实践名称：迷宫问题

学号：10245101434

一、实验任务

迷宫问题

无测评机，需要Presentation

求解

分别用BFS, DFS, Dijkstra, A star解决迷宫问题

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含0或1，其中0表示可以走的路，1表示不可通过的墙壁。

最初，有一个人位于左上角 $(1, 1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n, m) 处，至少需要移动多少次。

数据保证 $(1, 1)$ 处和 (n, m) 处的数字为0，且一定至少存在一条通路。

输入格式

第一行包含两个整数 n 和 m 。

接下来 n 行，每行包含 m 个整数（0或1），表示完整的二维数组迷宫。

输出格式

输出一个整数，表示从左上角移动至右下角的最少移动次数。

数据范围

$1 \leq n, m \leq 100$

二、实验过程

1. 四种算法概览

对比维度	DFS	BFS	Dijkstra	A*
🕒 时间复杂度	$O(V+E)$	$O(V+E)$	$O(V^2\log V)$	$O((V+E)\log V)$
🏠 空间复杂度	$O(h)$	$O(V)$	$O(V)$	$O(V)$
🏆 最优解保证	非最优	最优	最优	最优

注意：DFS 无法找到最短路径，不符何题目要求！

解决方案：在普通的 DFS 算法上进行修改，实现一个修改版的 DFS（Powerful DFS）：

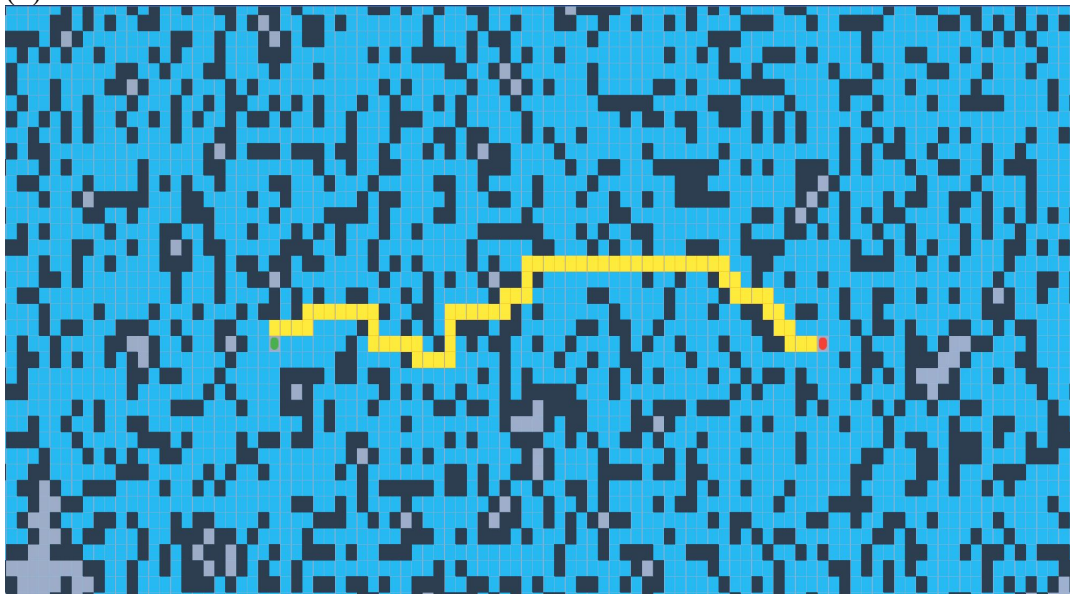
1. 使用递归 DFS 枚举所有从起点到终点的路径
2. 使用回溯（backtracking）避免当前路径中的循环
3. 比较所有路径长度，返回最短的那条

2. 实验设计

- 对所有算法应用同一套迷宫，迷宫大小为 100×100
- 设计三个维度的评价指标：访问节点数衡量搜索效率（越少越好），路径长度评估解的质量（越短越好），运行时间反映实际性能（毫秒级）
- 对每一种算法跑十次取平均值

3. 实验结果

(1). Powerful DFS



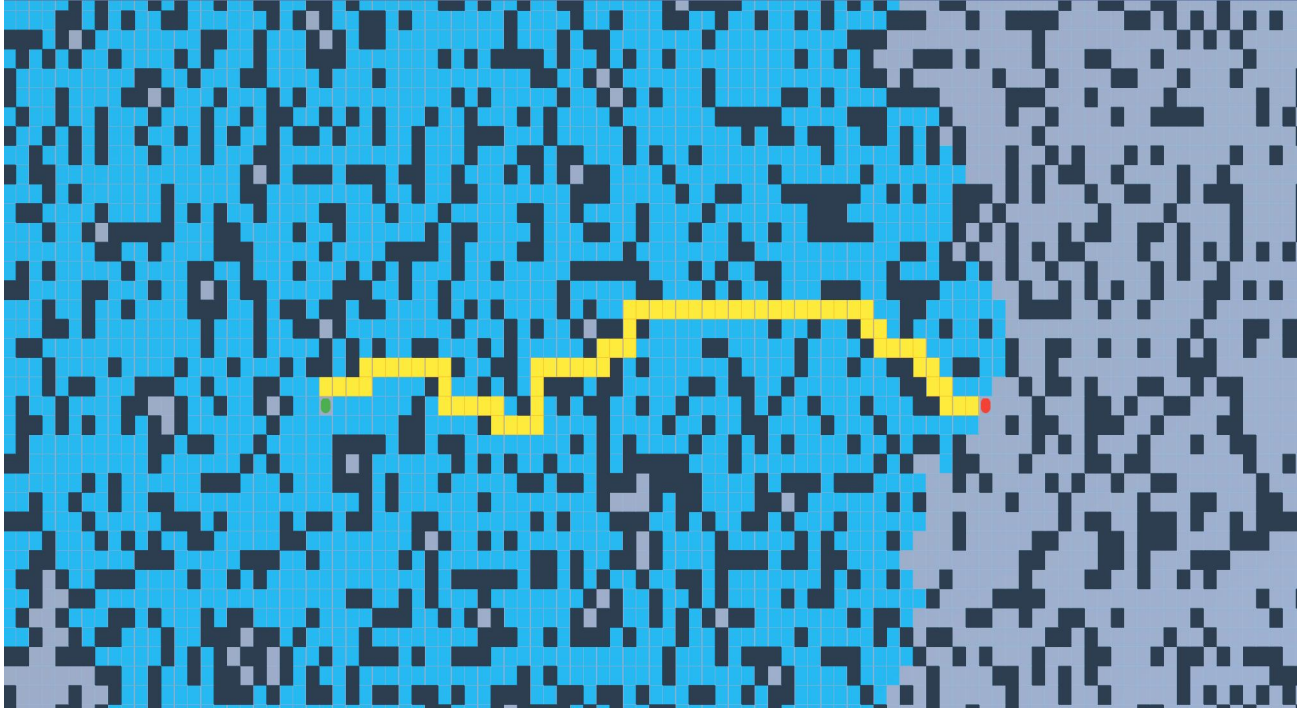
（注：迷宫未截全）

访问节点数: 6638

路径长度: 66

花费时间: 542.62ms

(2). BFS



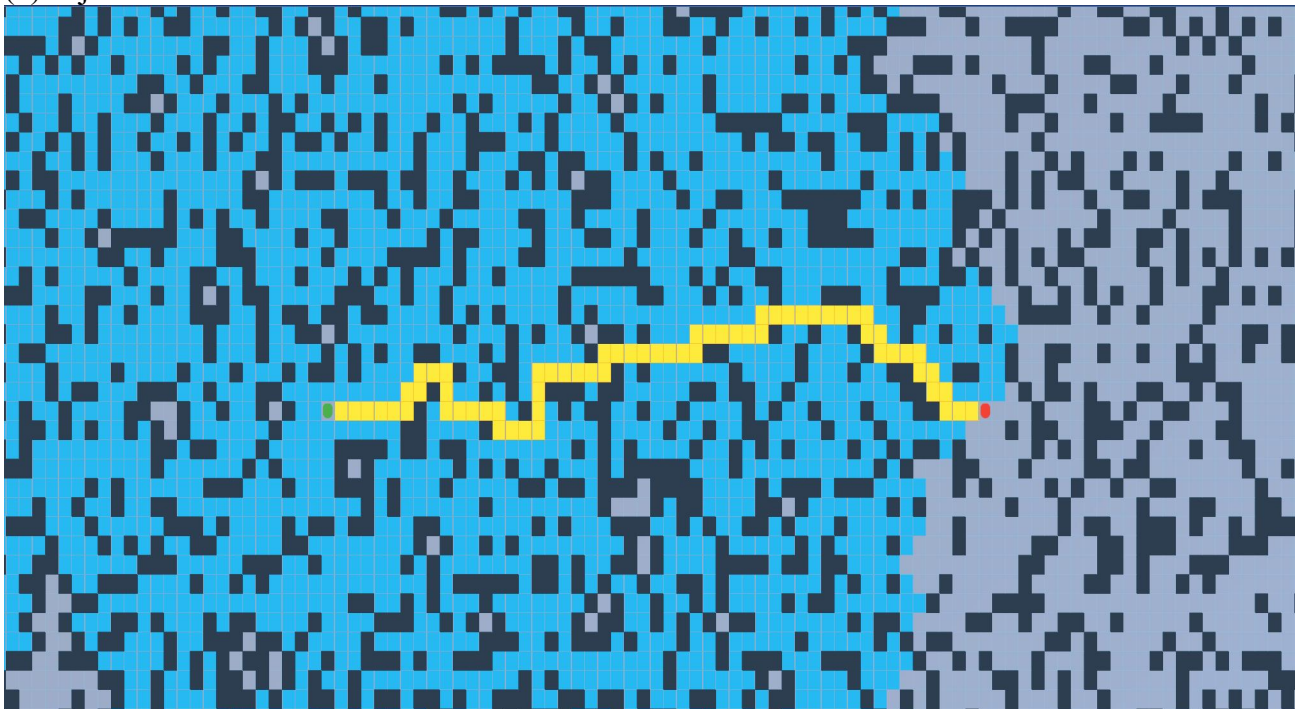
(注: 迷宫未截全)

访问节点数: 3820

路径长度: 66

花费时间: 2.34ms

(3). Dijkstra



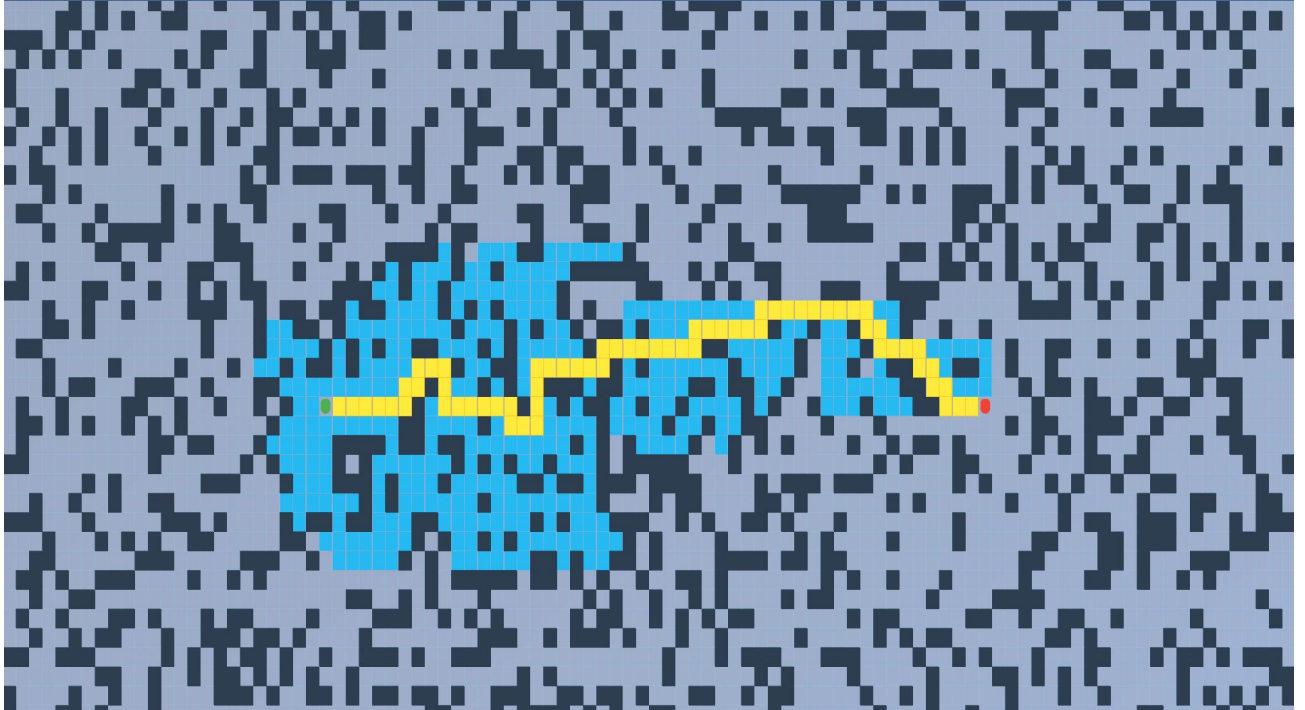
(注：迷宫未截全)

访问节点数：3820

路径长度：66

花费时间：572.22ms

(4). A*



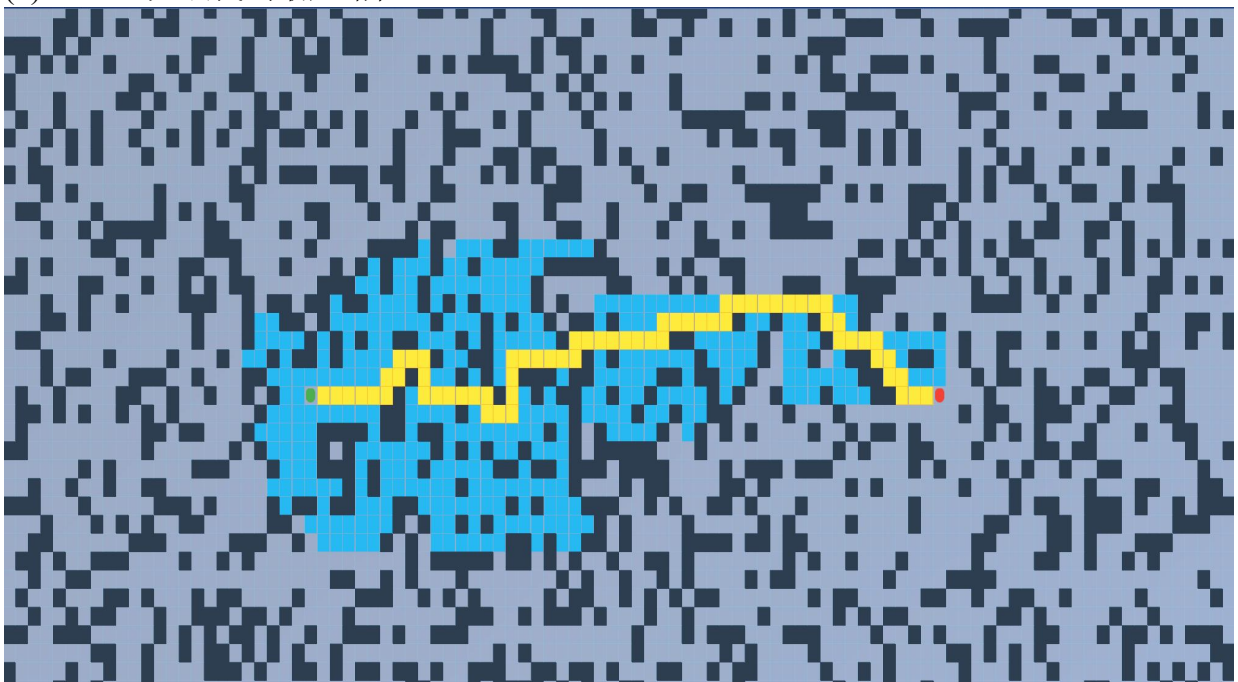
(注：迷宫未截全)

访问节点数：403

路径长度：66

花费时间：2.46ms

(5). DFS (无法找到最短路径!)



(注：迷宫未截全)

访问节点数：2689

路径长度：1234

花费时间：1.86ms

4. 对这五种算法的总结

	访问节点数	路径长度	花费时间
Powerful DFS	6638	66	542.62ms
BFS	3820	66	2.34ms
Dijkstra	3820	66	572.22ms
A*	403	66	2.46ms
DFS	2689	1234	1.86ms

(1). 核心结论总览

- 最短路径最优：BFS、Dijkstra、A*、Powerful DFS，均找到长度 66 最优最短路径
- 普通 DFS 路径极长 1234，完全不是最短路径，盲目深度搜索无最优性
- A* 综合性能优于其他算法：访问节点极少、路径最短、速度极快

(2). 逐项指标对比

- 路径长度（最优性）
 - BFS、Dijkstra、Powerful DFS、A*：路径长度 = 66 → 全局最短最优路径
 - 普通 DFS：路径长度 = 1234。DFS 优先一路走到底，不回头择优，极易走死胡同、绕远路，无权图不保证最短路径
- 访问节点数（搜索效率 / 空间开销）
 - A*：仅 403 个节点，启发式搜索极度高效，几乎不盲目遍历
 - BFS：3820 节点，逐层扩散，无多余遍历但广度开销大
 - Dijkstra：3820 节点，和 BFS 一致，但加权排序计算极耗时间
 - DFS：2689 节点，遍历较多且结果错误
 - Powerful DFS：6638 节点，盲目深度遍历开销最大
- 花费时间（运行效率）
 - 最快：普通 DFS 1.86ms（只往下冲，不对比、不回溯择优）
 - BFS：2.34ms
 - A*：2.46ms → 三者速度几乎同一级别，极快
 - Powerful DFS：542.62ms，暴力全量搜索耗时高
 - Dijkstra：572.22ms，频繁堆排序、距离更新、松弛操作，计算开销巨大，时间远高于其他算法

5. 拓展对比分析

(1). BFS vs 双向 BFS

	BFS	双向 BFS
搜索半径	r	双圆半径 $r/2$
时间复杂度	$O(V + E)$	$O(V/2 + E/2)$
节点访问模式	单向波扩展	双向波相遇
访问节点数	3820	1878
路径长度	66	66
花费时间	2.34ms	2.8ms

• 注意到双向 BFS 花费的时间并没有达到想象中的 BFS 的一半，原因：

- 维护了更多的数据结构，代码实现开销更大
- 理论差距较小，不一定能很明显的反映在结果中

(2). A* vs Weighted A* vs Greedy Search

	A*	Weighted A*	Greedy Search
启发函数	$f(n) = g(n) + h(n)$	$f(n) = g(n) + w \cdot h(n)$	$f(n) = h(n)$
评估策略	实际代价与启发估计平衡	类似 A*，通过权重系数偏向启发式	仅依赖启发函数估计
搜索范围	遍历节点较多，探索全面	访问节点较少，搜索较激进	访问节点最少，搜索激进
访问节点数	3820	136	107
路径长度	66	72	78
花费时间	2.34ms	1.22ms	0.86ms

• 三项数据直观对比

• 访问节点数：贪心 (107) < 加权 A*(136) < 标准 A*(3820)。权重越大，越偏向贪心，搜索范围越小，遍历节点越少

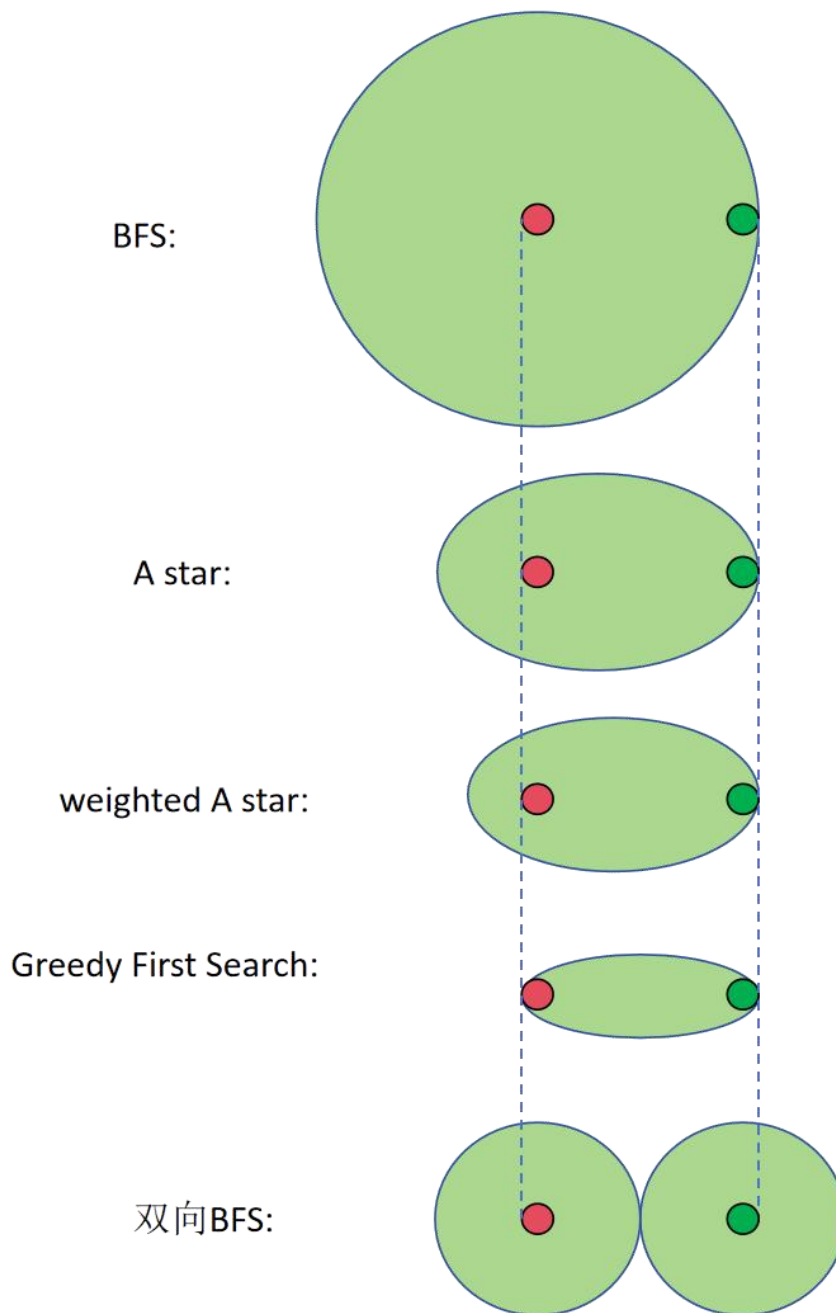
• 路径长度（路径优劣 / 最优性）：标准 A*(66) < 加权 A*(72) < 贪心 (78)。标准 A 严格最短；加权 A 次优；贪心路径最长，不是最短路径

• 运行耗时：贪心 (0.86ms) < 加权 A*(1.22ms) < 标准 A*(2.34ms)。启发权重越高，计算越简单，搜索越快

• 算法规律总结

• 权重 w 越大 $\rightarrow$ 搜索越快、节点越少、路径越长
 加权 A* 是速度与最优性的折中算法
 $w=0 \rightarrow$ 退化成 Dijkstra
 极大 $\rightarrow$ 退化成贪心搜索

6. 搜索范围可视化分析



(1). BFS

搜索形状：以起点为圆心的圆形（无向图 / 均匀网格中，按“到起点的距离”一层层向外扩散）。

(2). A*

搜索形状：明显被“拉向终点”的椭圆，比 BFS 的圆形范围小很多。

(3). Weighted A*

搜索形状：比标准 A* 更扁、更“瘦”的椭圆，范围进一步缩小，更贴近终点方向。

(4). Greedy First Search

搜索形状：被“极度拉向终点”的细长椭圆，探索范围是所有单方向算法里最小的。

(5). 双向 BFS

搜索形状：两个以起点、终点为圆心的圆形，分别从两端同时向外扩散，直到相遇。

7. 结论

- BFS 是等权图最优选择，保证最短路径
- Bidirectional BFS 理论上性能提升 2-4 倍，但实际测试中不一定能达到
- Dijkstra 存在性能陷阱，资源浪费 20-100 倍
- A*算法平衡最优性与执行效率，Weighted A*更快但牺牲最优性，Greedy Best-First Search 同理
- DFS 不保证找到最短路径，PowerfulDFS 搜索效率极低

三、总结

本次实验以 100×100 统一迷宫为测试环境，选取普通 DFS、改进型 Powerful DFS、BFS、Dijkstra、A 五种基础算法，搭配双向 BFS、加权 A、贪心搜索拓展算法，以访问节点数、路径长度、运行时间为核心指标，重复十次取均值开展对比测试。

实验验证了各算法特性：普通 DFS 无法获取最短路径，Powerful DFS 虽能找到最优解但遍历节点最多、耗时极长；BFS、Dijkstra 均可得到最短路径，前者运行高效，后者因堆排序操作性能极差；A 算法凭借启发式函数，在保证最优路径的同时实现了极高效率。拓展测试表明，双向 BFS 可减少节点访问量，但受实现开销影响未达理论提速效果；加权 A、贪心搜索速度更快、搜索范围更小，但会牺牲路径最优性。

通过实验，我清晰掌握了各类搜索算法的原理、优劣与适用场景，理解了启发式搜索对效率的提升作用，以及最优性与运行效率的平衡关系。